

Rapport TP2 – Deep Learning

PyTorch : prétraitement des données, gestion mémoire et régularisation

September 26, 2026

Contents

1	Introduction	2
2	Récupération du jeu de données	2
3	Prétraitement des données et fuite de données	2
3.1	Problème posé par l'application de <code>StandardScaler()</code> avant le split	2
3.2	Bonne pratique	2
3.3	Jeu de données trop volumineux pour la RAM	3
4	Régularisation L1 et L2	3
4.1	Effet d'une régularisation L1 forte	3
4.2	Régularisation L2 automatique avec PyTorch	3
4.3	Différence entre L1 et L2	3
4.4	Comparaison expérimentale	3
5	Comparaison des optimiseurs	4
5.1	Protocole	4
5.2	Résultats	5
5.3	Analyse	5
6	Métriques d'évaluation : Précision, Rappel et courbe ROC	6
6.1	Définitions	6
6.2	Précision ou rappel dans un contexte médical ?	6
6.3	Utilité de l'aire sous la courbe ROC (AUC)	6

1 Introduction

Ce TP porte sur trois aspects pratiques essentiels de l'entraînement de modèles de deep learning avec PyTorch :

- les risques de fuite de données (*data leakage*) lors du prétraitement,
- la gestion de jeux de données trop volumineux pour tenir en mémoire,
- l'effet de la régularisation L1 et L2 sur l'apprentissage d'un réseau de neurones.

2 Récupération du jeu de données

Le jeu de données `cardio_train.csv` a été transféré depuis la machine locale vers le cluster de calcul via `scp`, puis vérifié sur le serveur (figures 1 et 2).

```
(base) maroua@DESKTOP-83R5LLR:~$ scp "/mnt/c/Users/HA/Downloads/cardio_train.csv" tsp-client:~/TP1/TP2/data/
mzoubir@ssh.imtbs-tsp.eu's password:
mzoubir@157.159.104.128's password:
cardio_train.csv                                     100% 2873KB   4.1MB/s   00:00
(base) maroua@DESKTOP-83R5LLR:~$
```

Figure 1: Transfert du dataset vers le cluster via `scp`

```
[compute] [job 3398] mzoubir@starfighter-slurm-node-06-1:~/TP1/TP2/data (main)
$ ls
cardio_train.csv
```

Figure 2: Vérification de la présence du fichier sur le cluster

3 Prétraitement des données et fuite de données

3.1 Problème posé par l'application de `StandardScaler()` avant le split

Appliquer `StandardScaler()` sur l'ensemble du jeu de données *avant* de réaliser le split entraîne une fuite de données (*data leakage*). En effet, le `scaler` calcule la moyenne et l'écart-type à partir de la totalité des données, y compris celles qui seront ensuite utilisées pour la validation et le test.

Les informations provenant des ensembles de validation et de test influencent donc indirectement le prétraitement des données d'entraînement. Cela peut conduire à une estimation trop optimiste des performances du modèle, car ce dernier bénéficie indirectement d'informations issues de données qu'il n'est censé jamais voir pendant l'entraînement.

3.2 Bonne pratique

La démarche correcte consiste à :

1. séparer d'abord les données en ensembles *train*, *validation* et *test* ;
2. ajuster (`fit`) le `StandardScaler` uniquement sur les données d'entraînement ;
3. appliquer (`transform`) ce même `scaler` aux ensembles de validation et de test.

Ainsi, les ensembles de validation et de test restent totalement indépendants du processus d'apprentissage, ce qui garantit une évaluation fiable des performances du modèle.

3.3 Jeu de données trop volumineux pour la RAM

Si le jeu de données est trop volumineux pour tenir entièrement en mémoire (par exemple, 500 Go de données tabulaires), il est préférable d'utiliser la classe `IterableDataset` de PyTorch plutôt que `Dataset`.

`IterableDataset` permet de lire et de traiter les données progressivement, par exemple par lots ou par morceaux depuis le disque, sans charger l'intégralité du jeu de données en RAM. Cette approche est particulièrement adaptée aux très gros volumes de données, puisqu'elles peuvent être parcourues de manière séquentielle au fur et à mesure des besoins du `DataLoader`.

4 Régularisation L1 et L2

4.1 Effet d'une régularisation L1 forte

Avec $\lambda_1 = 0.1$ et $\lambda_2 = 0$, la régularisation L1 est très forte. On observe que l'apprentissage est fortement pénalisé : la loss reste élevée ou diminue difficilement, et l'accuracy peine à progresser.

Une pénalité L1 trop importante pousse fortement les poids du réseau vers zéro. Le modèle devient alors trop contraint pour apprendre correctement les relations présentes dans les données. Ce phénomène est appelé **sous-apprentissage** (*underfitting*).

4.2 Régularisation L2 automatique avec PyTorch

L'optimiseur `SGD` possède l'argument `weight_decay`, qui permet d'appliquer automatiquement une régularisation de type L2, sans avoir à calculer manuellement la pénalité dans la boucle d'entraînement.

4.3 Différence entre L1 et L2

La régularisation **L1** pénalise la somme des valeurs absolues des poids :

$$\mathcal{L}_{L1} = \sum_i |w_i|$$

Elle tend à pousser certains poids exactement vers zéro, ce qui rend le réseau plus parcimonieux (*sparse*).

La régularisation **L2** pénalise la somme des carrés des poids :

$$\mathcal{L}_{L2} = \sum_i w_i^2$$

Elle tend à réduire l'amplitude des poids et à les maintenir petits, sans nécessairement les annuler.

Ainsi, L1 favorise la parcimonie (davantage de poids nuls) tandis que L2 réduit globalement l'amplitude des poids sans les annuler.

4.4 Comparaison expérimentale

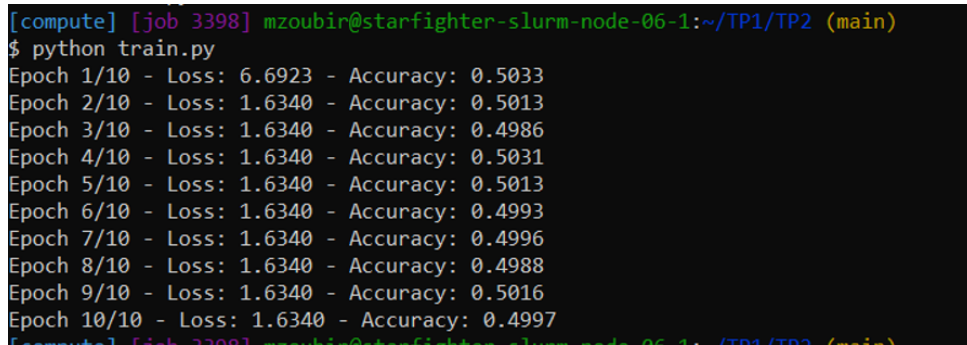
Le tableau 1 résume les résultats obtenus pour les deux configurations testées.

Table 1: Comparaison des configurations de régularisation

Configuration	Loss début → fin	Accuracy début → fin
$\lambda_1 = 0.1, \lambda_2 = 0$	6.6923 → 1.6340	50.33% → 49.97%
$\lambda_1 = 10^{-4}, \lambda_2 = 10^{-3}$	0.8409 → 0.7392	63.28% → 67.54%

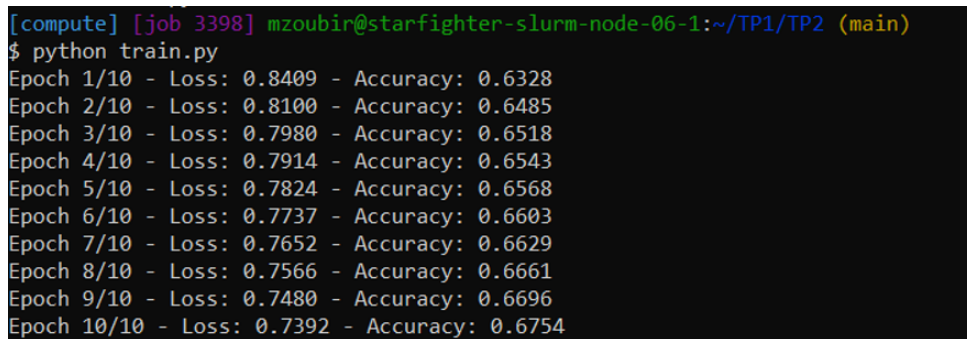
Sorties d'entraînement

Les figures 3 et 4 montrent les sorties terminal de `train.py` pour chacune des deux configurations.



```
[compute] [job 3398] mzoubir@starfighter-slurm-node-06-1:~/TP1/TP2 (main)
$ python train.py
Epoch 1/10 - Loss: 6.6923 - Accuracy: 0.5033
Epoch 2/10 - Loss: 1.6340 - Accuracy: 0.5013
Epoch 3/10 - Loss: 1.6340 - Accuracy: 0.4986
Epoch 4/10 - Loss: 1.6340 - Accuracy: 0.5031
Epoch 5/10 - Loss: 1.6340 - Accuracy: 0.5013
Epoch 6/10 - Loss: 1.6340 - Accuracy: 0.4993
Epoch 7/10 - Loss: 1.6340 - Accuracy: 0.4996
Epoch 8/10 - Loss: 1.6340 - Accuracy: 0.4988
Epoch 9/10 - Loss: 1.6340 - Accuracy: 0.5016
Epoch 10/10 - Loss: 1.6340 - Accuracy: 0.4997
```

Figure 3: Sortie terminal – forte régularisation ($\lambda_1 = 0.1$)



```
[compute] [job 3398] mzoubir@starfighter-slurm-node-06-1:~/TP1/TP2 (main)
$ python train.py
Epoch 1/10 - Loss: 0.8409 - Accuracy: 0.6328
Epoch 2/10 - Loss: 0.8100 - Accuracy: 0.6485
Epoch 3/10 - Loss: 0.7980 - Accuracy: 0.6518
Epoch 4/10 - Loss: 0.7914 - Accuracy: 0.6543
Epoch 5/10 - Loss: 0.7824 - Accuracy: 0.6568
Epoch 6/10 - Loss: 0.7737 - Accuracy: 0.6603
Epoch 7/10 - Loss: 0.7652 - Accuracy: 0.6629
Epoch 8/10 - Loss: 0.7566 - Accuracy: 0.6661
Epoch 9/10 - Loss: 0.7480 - Accuracy: 0.6696
Epoch 10/10 - Loss: 0.7392 - Accuracy: 0.6754
```

Figure 4: Sortie terminal – $\lambda_1 = 10^{-4}$, $\lambda_2 = 10^{-3}$

Observation sur l'apprentissage avec régularisation

Avec une régularisation L1 faible ($\lambda_1 = 10^{-4}$) et L2 ($\lambda_2 = 10^{-3}$), la loss diminue progressivement de 0,8409 à 0,7392 et la précision augmente de 63,28 % à 67,54 %. Le modèle continue donc à apprendre au cours des epochs.

En revanche, avec une forte régularisation L1 ($\lambda_1 = 0,1$ et $\lambda_2 = 0$), la loss passe de 6,6923 à 1,6340 dès les premières epochs, puis reste stable. La précision reste également proche de 50 % (49,97 % à la fin), ce qui montre que le modèle apprend très peu dans cette configuration. Cette forte pénalisation L1 contraint fortement les paramètres du modèle et peut conduire à du sous-apprentissage.

5 Comparaison des optimiseurs

5.1 Protocole

Afin de comparer l'influence du choix de l'optimiseur sur la convergence de l'entraînement, quatre configurations ont été testées sur le jeu de données *cardio*, avec un taux d'apprentissage identique ($lr = 0.001$) :

- SGD (sans momentum) ;
- SGD avec momentum ;
- RMSprop ;

- Adam.

La figure 5 présente l'évolution de la *training loss* pour chacune des quatre configurations, suivie via TensorBoard.

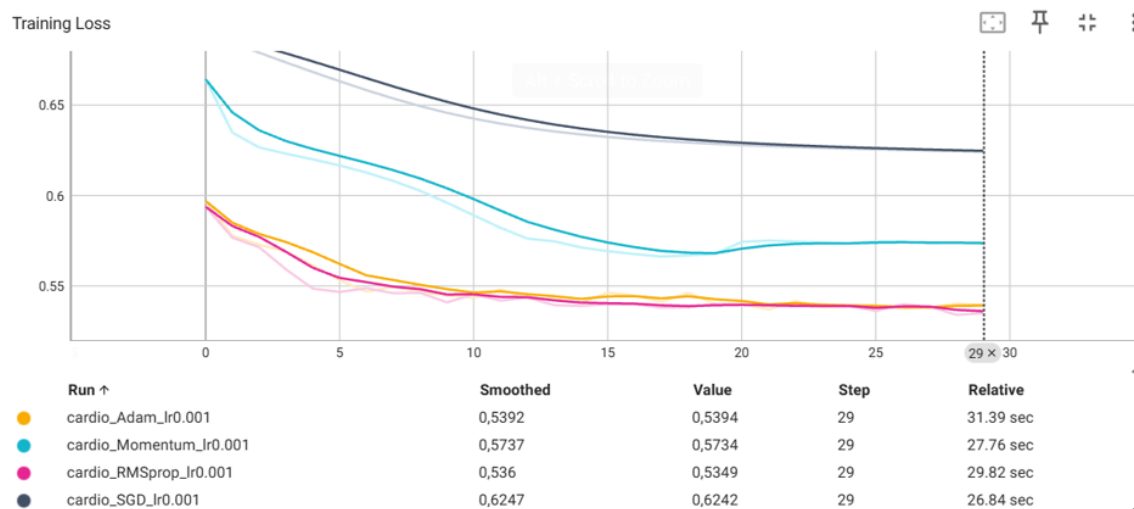


Figure 5: Training loss pour les quatre optimiseurs (cardio, $lr = 0.001$)

5.2 Résultats

Le tableau 2 résume la loss finale obtenue à la fin de l'entraînement (step 29) pour chaque optimiseur.

Table 2: Loss finale par optimiseur (step 29)

Optimiseur	Loss finale
RMSprop	0,5349
Adam	0,5394
Momentum	0,5734
SGD	0,6242

5.3 Analyse

Adam et RMSprop dominant nettement la comparaison : leurs courbes chutent rapidement dès les premières epochs et se stabilisent autour de 0,53–0,54. Ce comportement s'explique par le fait que ce sont des méthodes à taux d'apprentissage adaptatif, qui ajustent le pas d'apprentissage individuellement pour chaque paramètre en fonction de l'historique des gradients.

SGD (sans momentum) est l'optimiseur le plus lent : sa loss reste élevée (environ 0,62) même après 29 steps. Avec un taux d'apprentissage fixe de 0.001 et aucune forme d'accélération, l'optimiseur peine à progresser dans ce budget d'epochs.

SGD avec momentum se situe entre les deux extrêmes : le momentum accélère la convergence par rapport au SGD pur, grâce à l'inertie accumulée qui permet de sortir plus facilement des zones plates de la fonction de perte. On observe cependant une légère remontée de la loss autour du step 18–20, qui peut s'expliquer par une oscillation autour d'un minimum local : le momentum accumulé pousse alors le pas au-delà de l'optimum avant que la trajectoire ne se corrige.

En résumé, sur cet horizon d'entraînement limité, les méthodes adaptatives (Adam, RM-Sprop) convergent nettement plus vite et atteignent une loss plus basse que les méthodes à taux d'apprentissage fixe (SGD, SGD + momentum).

6 Métriques d'évaluation : Précision, Rappel et courbe ROC

6.1 Définitions

La **précision** (*Precision*) mesure, parmi toutes les prédictions positives faites par le modèle, la proportion qui sont réellement positives :

$$\text{Precision} = \frac{TP}{TP + FP}$$

où TP (*True Positives*) désigne les vrais positifs et FP (*False Positives*) les faux positifs. Une précision élevée signifie que le modèle se trompe peu lorsqu'il prédit la classe positive.

Le **rappel** (*Recall*), aussi appelé sensibilité, mesure, parmi tous les cas réellement positifs, la proportion que le modèle parvient effectivement à détecter :

$$\text{Recall} = \frac{TP}{TP + FN}$$

où FN (*False Negatives*) désigne les faux négatifs. Un rappel élevé signifie que le modèle manque peu de cas positifs réels.

6.2 Précision ou rappel dans un contexte médical ?

Dans le contexte du diagnostic d'une maladie cardiovasculaire, il est préférable de **privilégier un modèle avec un fort rappel**, quitte à sacrifier un peu de précision.

En effet, un faux négatif (un patient réellement malade, mais classé comme sain) est bien plus coûteux qu'un faux positif : un patient malade non détecté ne recevra pas de traitement à temps, avec un risque vital, alors qu'un faux positif (patient sain classé malade à tort) entraînera au pire des examens complémentaires inutiles mais sans danger immédiat. Il est donc essentiel de minimiser les faux négatifs, c'est-à-dire de maximiser le rappel, même si cela conduit à davantage de faux positifs (donc une précision plus faible).

6.3 Utilité de l'aire sous la courbe ROC (AUC)

Les métriques comme la précision, le rappel ou l'accuracy sont calculées à un **seuil de décision fixe** (généralement 0.5) : elles ne renseignent que sur la performance du modèle pour ce seuil particulier, et peuvent donc masquer son comportement à d'autres seuils.

L'**aire sous la courbe ROC** (*Area Under the ROC Curve*, AUC) évalue au contraire la capacité du modèle à discriminer les classes positive et négative **indépendamment du seuil choisi**. La courbe ROC trace le taux de vrais positifs (rappel) en fonction du taux de faux positifs pour tous les seuils possibles, et l'AUC en résume l'aire globale :

- une AUC de 1 correspond à un classifieur parfait, qui sépare totalement les deux classes ;
- une AUC de 0.5 correspond à un classifieur aléatoire, sans pouvoir discriminant.

L'AUC est donc particulièrement utile pour comparer des modèles entre eux de manière globale, ou pour choisir a posteriori le seuil de décision le plus adapté au contexte applicatif (par exemple, un seuil plus bas que 0.5 pour privilégier le rappel dans un contexte médical), plutôt que de se limiter à l'évaluation d'un unique point de fonctionnement.